



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



Module

2

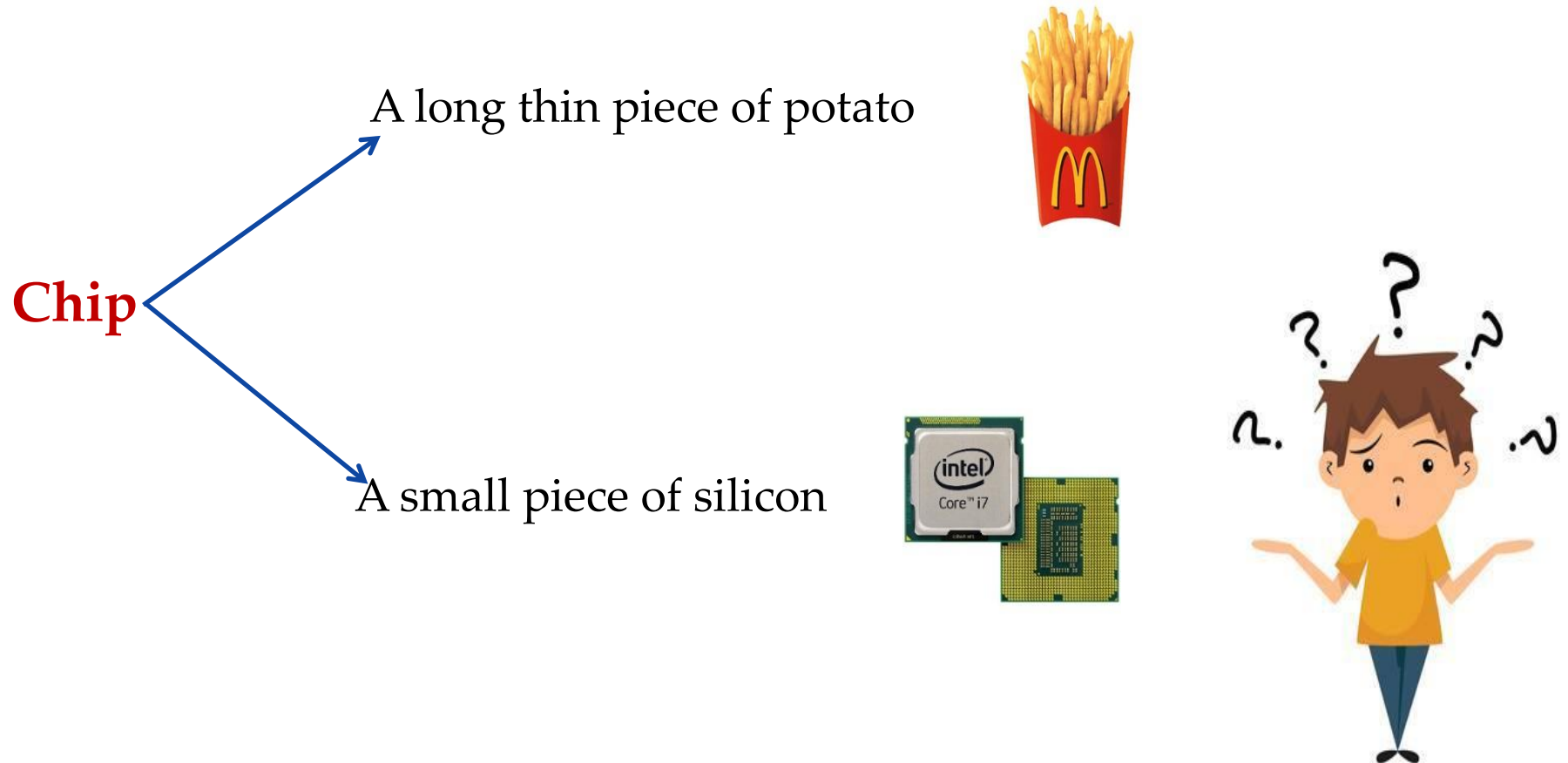
SYNTAX ANALYSIS

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

Elimination of Ambiguity

Ambiguity

- Ambiguity, is a word, phrase, or statement which contains **more than one meaning**.



AMBIGUOUS GRAMMAR

- A Grammar is ambiguous if any string derives

- More than one parse tree

(OR)

- More than one Left most Derivation

(OR)

- More than one Rightmost Derivation

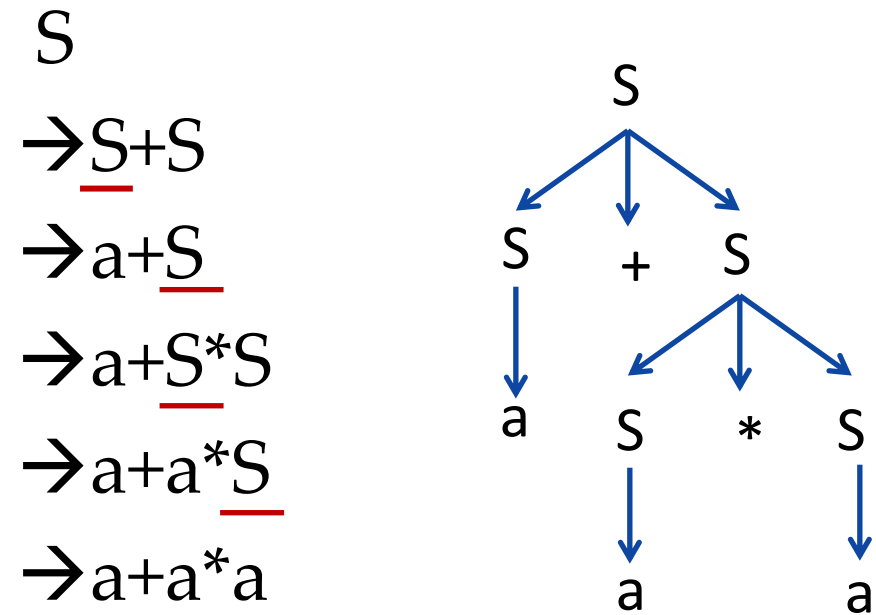
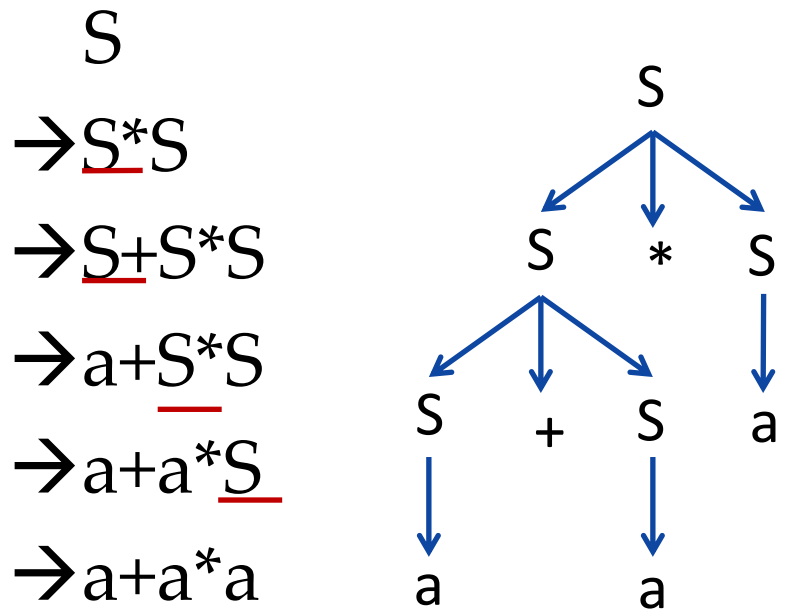
- **Drawbacks of Ambiguous Grammar**

- Parsing Complexity
 - Affects other phases

AMBIGUOUS GRAMMAR - REASONS

- Three main Reasons leads to Ambiguity
 - **Precedence**
 - **Associativity**
 - **Dangling else**
- **Ambiguity can be eliminated by**
 - Rewriting the grammar(unambiguous Grammar) or
 - Use ambiguous grammar with additional rules to resolve ambiguity

- Ambiguous grammar is one that produces more than one leftmost or more than one rightmost derivation for the same sentence.
- Grammar: $S \rightarrow S+S \mid S*S \mid (S) \mid a$ Output string: $a+a*a$



- Here, Two leftmost derivation for string $a+a*a$ is possible hence, above grammar is ambiguous.

Exercise: Ambiguous Grammar

Check Ambiguity in following grammars:

1. $S \rightarrow aS \mid Sa \mid \epsilon$ (output string: aaaa)
2. $S \rightarrow aSbS \mid bSaS \mid \epsilon$ (output string: abab)
3. $S \rightarrow SS+ \mid SS^* \mid a$ (output string: aa+a*)
4. $\langle \text{exp} \rangle \rightarrow \langle \text{exp} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$
 $\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{letter} \rangle \mid \langle \text{letter} \rangle$
 $\langle \text{letter} \rangle \rightarrow a \mid b \mid c \mid \dots \mid z$ (output string: a+b*c)
5. Prove that the CFG with productions:
 $S \rightarrow a \mid Sa \mid bSS \mid SSb \mid SbS$ is ambiguous
(Hint: consider any output string)

Converting Ambiguous Grammar into Unambiguous Grammar

- An ambiguous grammar may be converted into an unambiguous grammar by implementing

ASSOCIATIVITY

- It represents the direction in which the expression to be evaluated
- **Two types of Associativity**
 1. Left to Right
 2. Right to Left

OPERATOR PRECEDENCE

- When more than one operator presents in an expression which one to be evaluated first is determined by **Operator Precedence**

1. Associativity

- If the **same precedence operators** are in production, then we will have to consider the **associativity**.
 - If the **associativity is left to right**, then we have to prompt a **left recursion** in the production. The parse tree will also be left recursive and grow on the left side.

$+$, $-$, $*$, $/$ are left associative operators.

- Example : Consider $a + b + c$

$(a + b) + c$

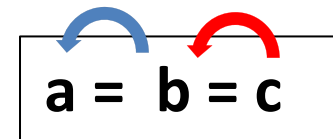


$a + (b + c)$



- If the **associativity is right to left**, then we have to prompt the **right recursion** in the productions. The parse tree will also be right recursive and grow on the right side.

$\wedge$, $=$ is a right associative operator.

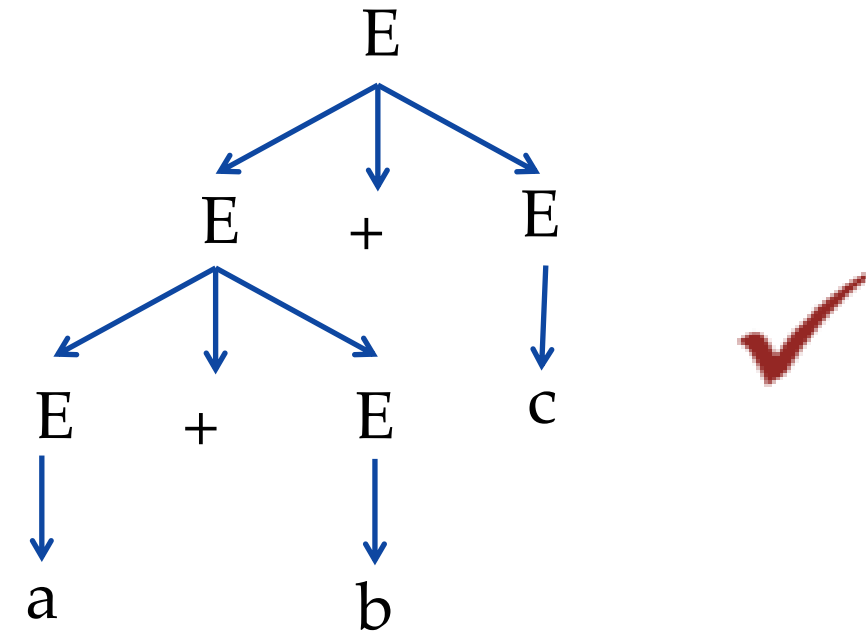
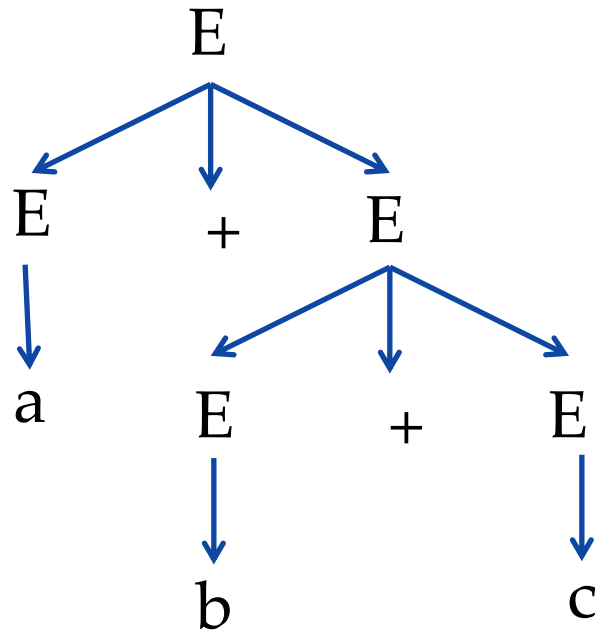


(i) Left Associate Grammar

- Consider the grammar

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow a / b / c \end{aligned}$$

Given String : **a + b + c**



- As per left associate grammar, the second parse tree is correct as it is evaluated from left to right

- Left associate operator introduces **left recursion** in production to eliminate ambiguity, by replacing **the left most non-terminal E in the right side of the production with another random variable, say T**

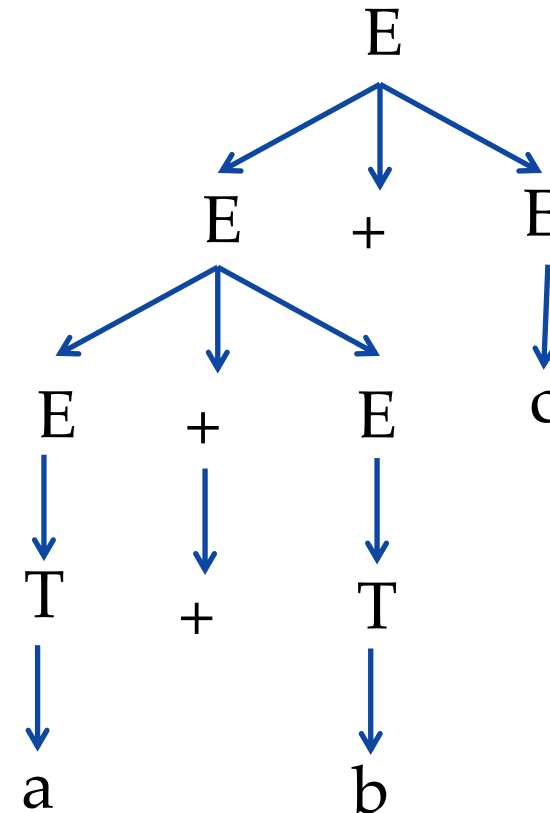
- Consider the grammar

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow a / b / c \end{aligned}$$

Given String : **a + b + c**

- The above grammar can be rewritten as

$$\begin{aligned} E &\rightarrow E + T / T \\ T &\rightarrow a / b / c \end{aligned}$$

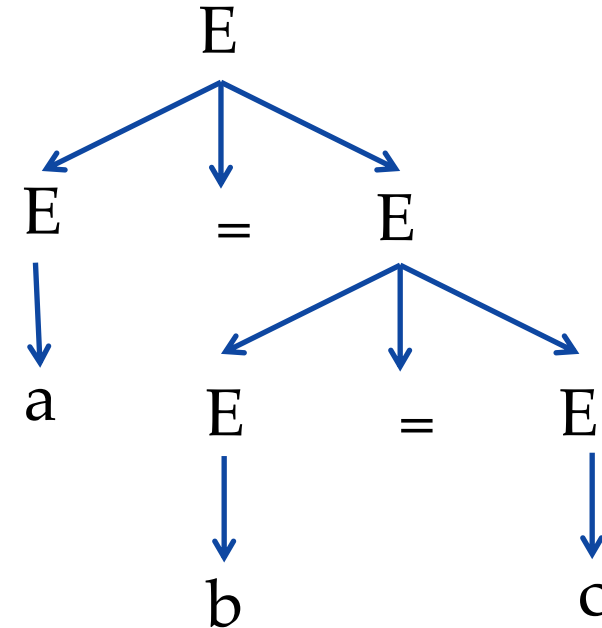
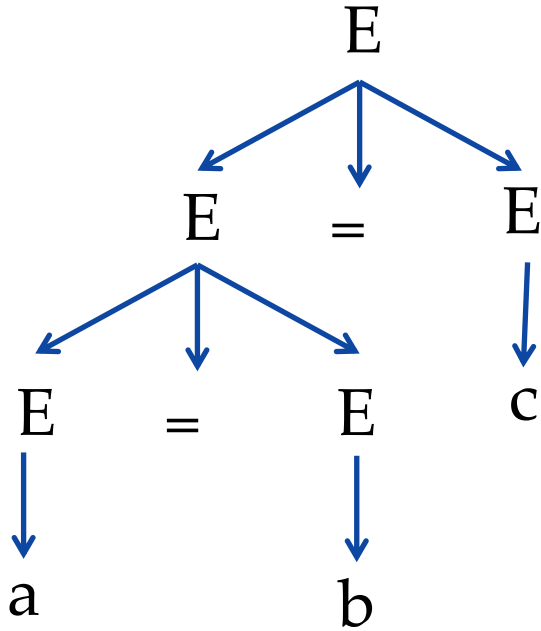


(ii) Right Associate Grammar

- Consider the grammar

$$\begin{aligned} E &\rightarrow E = E \\ E &\rightarrow a / b / c \end{aligned}$$

Given String : **a = b = c**



- As per right associate grammar, the second parse tree is correct as it is evaluated from right to left

- Right Associate Grammar introduces *right recursion* in production to eliminate ambiguity

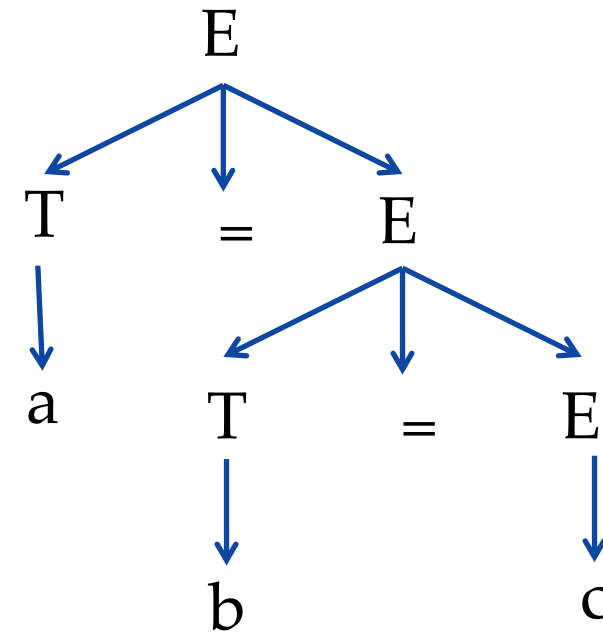
- Consider the grammar

$$\begin{aligned} E &\rightarrow E = E \\ E &\rightarrow a / b / c \end{aligned}$$

Given String : **a = b = c**

- The above grammar can be rewritten as

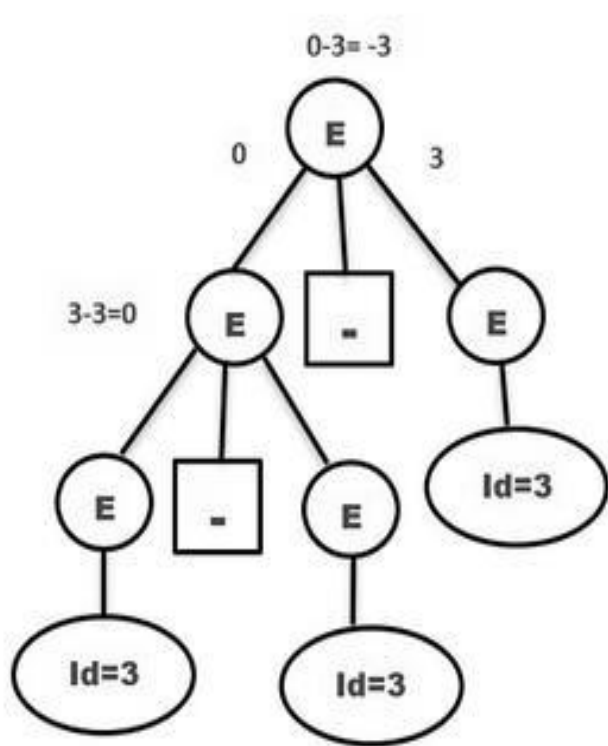
$$\begin{aligned} E &\rightarrow T = E / T \\ T &\rightarrow a / b / c \end{aligned}$$



Example 2: Consider the ambiguous grammar

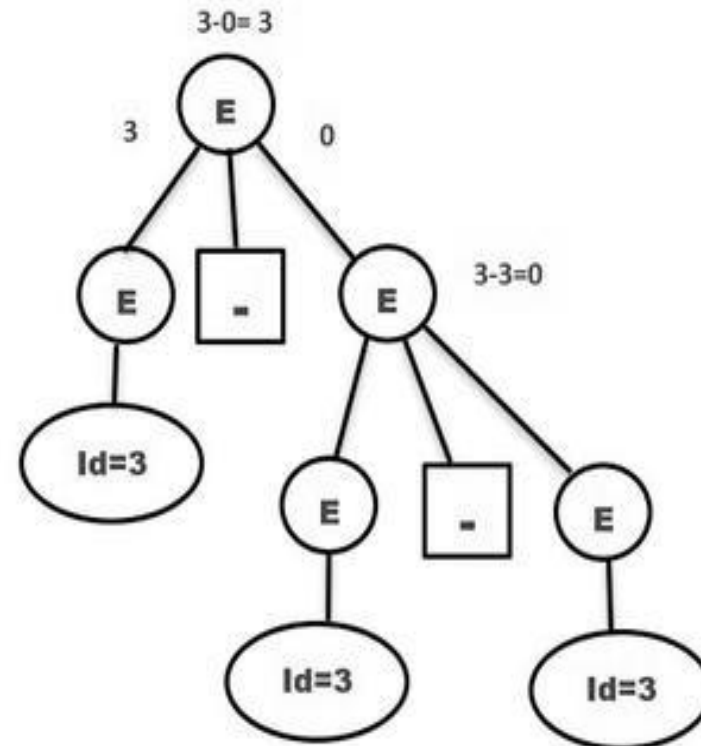
$E \rightarrow E-E \mid id$

- Derive the string **id-id-id** and consider **id=3**



Left Associative

$$((3-3)-3) = (0-3) = -3$$



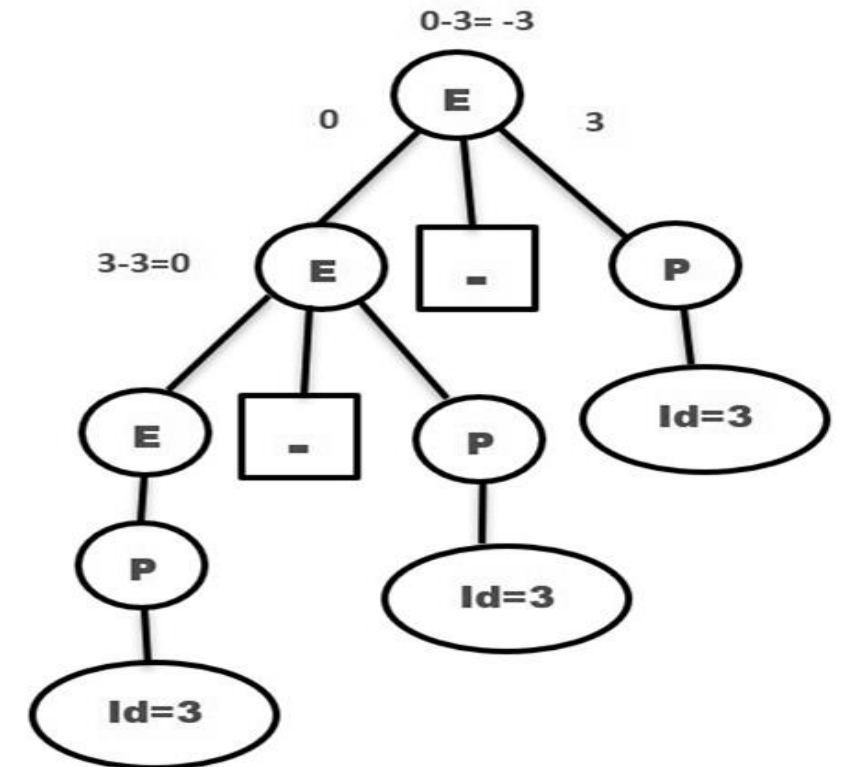
Right Associative

$$(3-(3-3)) = (3-0) = 3$$

- The above grammar can be rewritten as

$E \rightarrow E - P \mid P$

$P \rightarrow id$



3. Show that the following grammar is ambiguous grammar for the given string and remove ambiguity from the given grammar:

$$S \rightarrow S * S \mid a \quad \text{String, } w = a * a * a$$

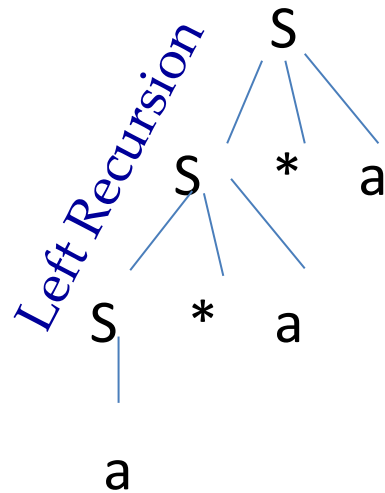
Soln:

$$\text{LMD1: } S \rightarrow S * S \rightarrow a * S \rightarrow a * S * S \rightarrow a * a * a$$

$$\text{LMD2: } S \rightarrow S * S \rightarrow S * S * S \rightarrow a * S * S \rightarrow a * a * S \rightarrow a * a * a$$

- Reconstructed grammar:

$$S \rightarrow S * a \mid a$$



2. Precedence

- If **different operators** are used, consider the **precedence of the operators**.
- The characteristics:
 - The ***level at which the production is present denotes the priority of the operator used.***
 - The ***production at higher levels will have operators with less priority.*** In the parse tree, the nodes which are at top levels or close to the root node will contain the lower priority operators.
 - The ***production at lower levels will have operators with higher priority.*** In the parse tree, the nodes which are at lower levels or close to the leaf nodes will contain the higher priority operators.
- Consider the expression $4 + 3 * 6$

* has higher precedence, so it has to be evaluated first

$$\begin{aligned} & \Rightarrow 4 + \underline{3 * 6} \\ & \Rightarrow 4 + 18 \\ & \Rightarrow 22 \end{aligned}$$



$$\begin{aligned} & \Rightarrow \underline{4 + 3} * 6 \\ & \Rightarrow 7 * 6 \\ & \Rightarrow 42 \end{aligned}$$



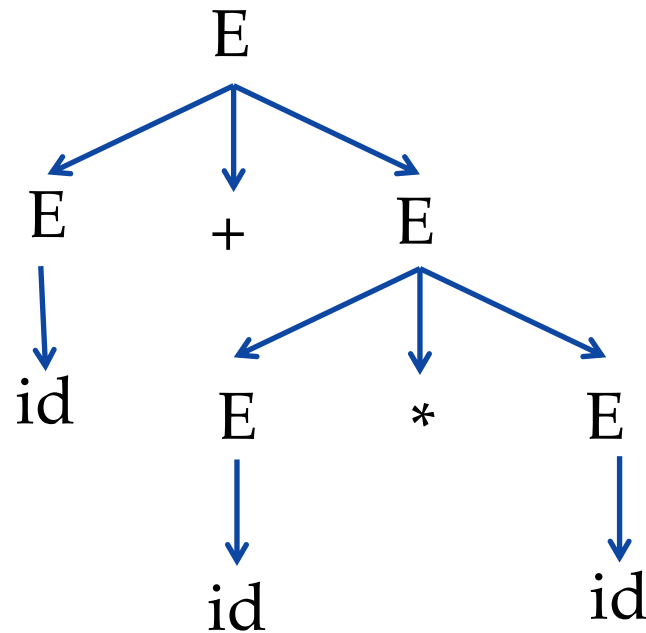
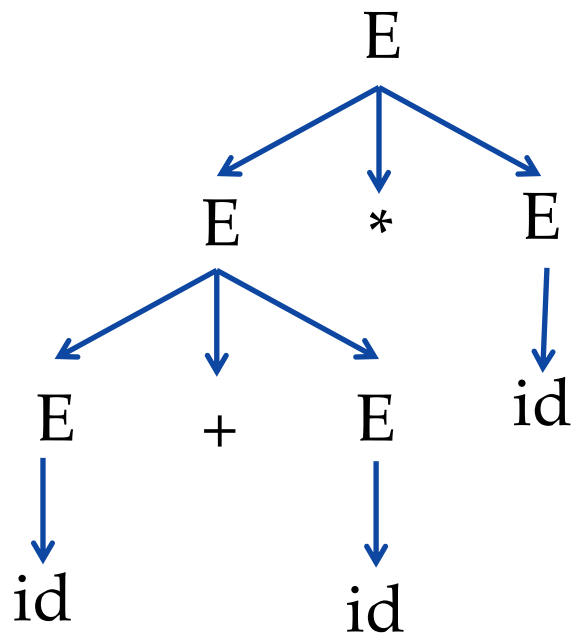
Example : Consider the grammar shown below, which has two different operators :

$$\begin{array}{l} E \rightarrow E + E \\ E \rightarrow E * E \\ E \rightarrow id \end{array}$$

Derive the string : **id + id * id**

Soln

Two parse trees for the string can be derived in 2 ways



- It is an ambiguous grammar, since it can create 2 parse tree for the given grammar.
- If operator precedence rule is not implemented then both the parse tree are correct.
- To eliminate ambiguity, the operator precedence rule has to be implemented.
- The unambiguous grammar will contain the productions having the highest priority operator ("*") at the lower level and vice versa.

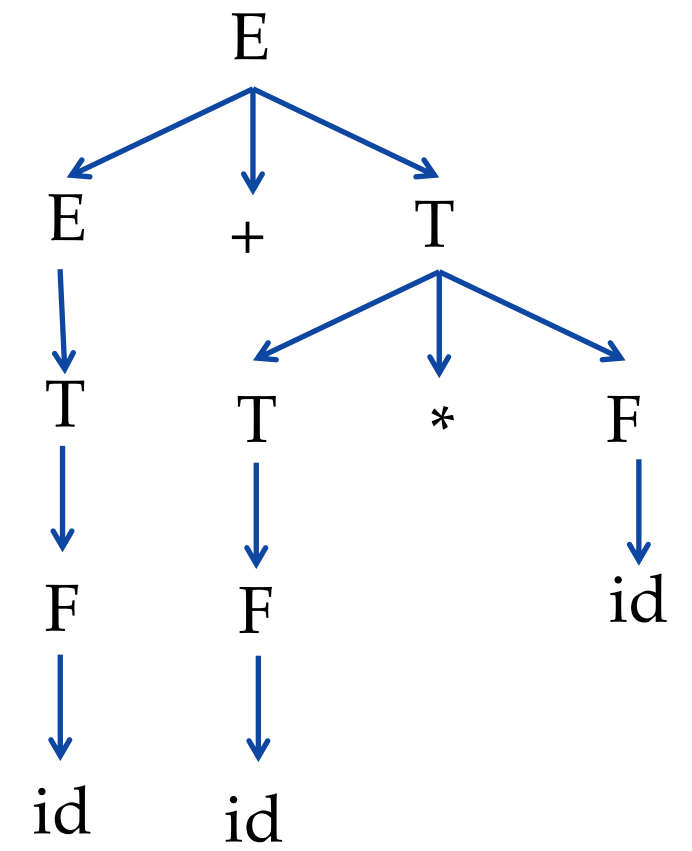
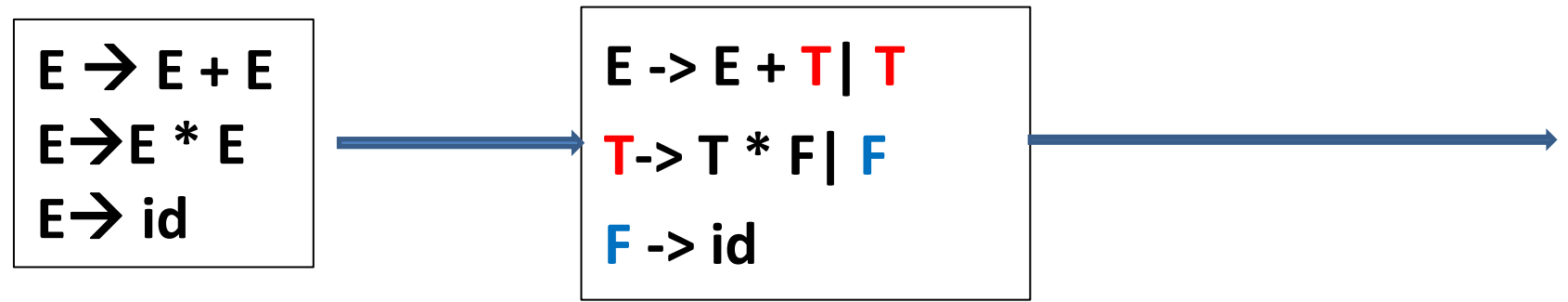
- The “+” having the least priority has to be at the upper level and has to wait for the result produced by the “*” operator which is at the lower level.
- Grammar has to be rewritten using the level of precedence concept.
- To convert ambiguous grammar to unambiguous, create levels of non terminal

E → First level of precedence

T → Second level of precedence

F → Generating basic units (terminal) in expression

- The unambiguous grammar for the given grammar:



- **E** is used for doing addition operations
- **T** is used to perform multiplication operations.
- They are independent and will maintain the precedence order in the parse tree.

- The unambiguous grammar for an expression having the operators $-$, $*$, $^$ is :

$E \rightarrow E - P \mid P$ // Minus operator is at higher level due to least priority and left associative.

$P \rightarrow P * Q \mid Q$ // Multiplication operator has more priority than $-$ and lesser than $^$ and left associative.

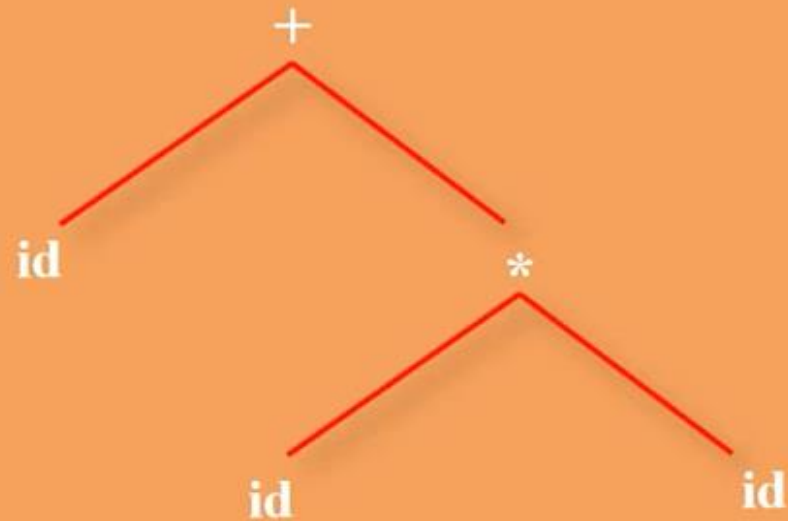
$Q \rightarrow R ^ Q \mid R$ // Exponent operator is at lower level due to highest priority and right associative.

$R \rightarrow id$

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(type){ list }	Compound literal(C99)	
2	++ --	Prefix increment and decrement	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(type)	Type cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10		Bitwise OR (inclusive or)	
11	&&	Logical AND	
12		Logical OR	
13	? :	Ternary conditional	Right-to-Left
14	=	Simple assignment	Left-to-right
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= =	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

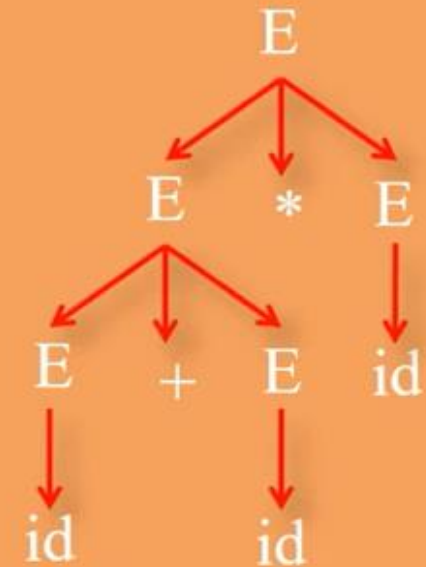
Syntax Tree

- ✓ In Syntax tree, interior nodes are operator, and leaves are Operand.
- ✓ It represents the abstract syntax tree of a program.
- ✓ Syntax tree contains only meaningful information.
- ✓ Example:
Grammar: $E \rightarrow E * E \mid E + E \mid id$
String: $id + id * id$ or $a + b * c$



Parse Tree

- ✓ In Parse tree, interior nodes are Nonterminals, and leaves are Terminals.
- ✓ It represents the concrete syntax of a program.
- ✓ Parse tree contains unusable information also.
- ✓ Example:
Grammar: $E \rightarrow E * E \mid E + E \mid id$
String: $id + id * id$ or $a + b * c$



LEFT RECURSION & LEFT FACTORING

LEFT RECURSION

- A grammar is said to be **left recursive** if it has a non terminal A such that there is a derivation $A \rightarrow A\alpha$ for some string α .

Left recursion elimination

$$A \rightarrow A\alpha \mid \beta \quad \longrightarrow \quad \begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \epsilon \end{array}$$

- The production is **left-recursive** if the leftmost symbol on the right side is the same as the non-terminal on the left side.

Examples: Left recursion elimination

1. Eliminate left recursion for the following grammar

$$\begin{array}{l} E \rightarrow E+T \mid T \\ T \rightarrow T*F \mid F \\ T \rightarrow (E) / \text{id} \end{array}$$
$$A \rightarrow A\alpha \mid \beta \longrightarrow \begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \epsilon \end{array}$$

$E \rightarrow E+T \mid T$ (RHS and LHS contain same non terminal symbol)

$$\begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \epsilon \end{array}$$

$T \rightarrow T*F \mid F$

$$\begin{array}{l} T \rightarrow FT' \\ T' \rightarrow *FT' \mid \epsilon \end{array}$$

Grammar after left recursion

$$\begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \epsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' \mid \epsilon \\ T \rightarrow (E) / \text{id} \end{array}$$

Left-Recursion

Problem-1: Consider the following grammar and eliminate left recursion-

$$A \rightarrow AA\alpha / \beta$$

Solution:

The grammar after eliminating left recursion is-

$$A \rightarrow \beta A'$$

$$A' \rightarrow A\alpha A' / \epsilon$$

Problem-2: Consider the following grammar and eliminate left recursion-

$$S \rightarrow SOS1S / 01$$

Solution:

The grammar after eliminating left recursion is-

$$S \rightarrow 01A$$

$$A \rightarrow OS1SA / \epsilon$$

Left-Recursion

Problem-3: Consider the following grammar and eliminate left recursion-

$A \rightarrow ABd / Aa / a$

$B \rightarrow Be / b$

Solution:

The grammar after eliminating left recursion is-

$A \rightarrow aA'$

$A' \rightarrow BdA' / aA' / \epsilon$

$B \rightarrow bB'$

$B' \rightarrow eB' / \epsilon$

Problem-4: Consider the following grammar and eliminate left recursion-

$E \rightarrow E + E / E \times E / a$

Solution:

The grammar after eliminating left recursion is-

$E \rightarrow aA$

$A \rightarrow +EA / \times EA / \epsilon$

Exercise: Left recursion

1. $A \rightarrow Abd \mid Aa \mid a$

$$B \rightarrow Be \mid b$$

2. $A \rightarrow AB \mid AC \mid a \mid b$

3. $S \rightarrow A \mid B$

$$A \rightarrow ABC \mid Acd \mid a \mid aa$$

$$B \rightarrow Bee \mid b$$

4. $\text{Exp} \rightarrow \text{Exp} + \text{term} \mid \text{Exp} - \text{term} \mid \text{term}$

Ex: 1

$$A \rightarrow Abd \mid Aa \mid a$$

$$B \rightarrow Be \mid b$$

Soln:

$$A \rightarrow a A'$$

$$A' \rightarrow bdA' \mid aA' \mid \epsilon$$

Ex: 2

$$A \rightarrow AB \mid AC \mid a \mid b$$

Soln:

$$A \rightarrow aA' \mid bA'$$

$$A' \rightarrow BA' \mid CA' \mid \epsilon$$

Ex: 3: $S \rightarrow A \mid B; A \rightarrow ABC \mid Acd \mid a \mid aa; B \rightarrow Bee \mid b$

Soln:

$$S \rightarrow A \mid B$$

$$A \rightarrow aA' \mid aaA'$$

$$A' \rightarrow BCA' \mid cdaA' \mid \epsilon$$

$$B \rightarrow bB'$$

$$B' \rightarrow eeB' \mid \epsilon$$

Ex: 4:

$$\text{Exp} \rightarrow \text{Exp} + \text{term} \mid \text{Exp} - \text{term} \mid \text{term}$$

Soln:

$$\text{Exp} \rightarrow \text{term} A'$$

$$A' \rightarrow + \text{term} A' \mid - \text{term} A' \mid \epsilon$$

LEFT FACTORING

- Left factoring is a **grammar transformation** that is useful for **producing a grammar suitable for predictive parsing**.
- Left factoring is a process by which the *grammar with common prefixes is transformed*.
- The common prefix may be a *terminal* or a *non-terminal* or a *combination of both*.
- Grammar in the following form will be considered to be having left factored

$$S \Rightarrow aX \mid aY \mid aZ$$

- S , X , Y , and Z are non-terminal symbols, and a is terminal.
- There are three different productions with the same non-terminal symbol on the left-hand side, and the productions have a common prefix a .

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \dots \dots \mid r_1 \mid r_2$$



$$A \rightarrow \alpha A' \mid r_1 \mid r_2$$

$$A' \rightarrow \beta_1 \mid \beta_2 \dots$$

Do left factoring in the following grammar.

$$S \rightarrow \mathbf{a}AB \mid \mathbf{a}CD$$

$$S \rightarrow \mathbf{a}S'$$

$$S' \rightarrow AB \mid CD$$

$$A \rightarrow \mathbf{xByA} \mid \mathbf{xByAzA} \mid \mathbf{a}$$

$$A \rightarrow \mathbf{xByAA'} \mid \mathbf{a}$$

$$A' \rightarrow \epsilon \mid \mathbf{zA}$$

$$A \rightarrow \mathbf{aAB} \mid \mathbf{aA} \mid \mathbf{a}$$

$$A \rightarrow \mathbf{aA'}$$

$$A' \rightarrow \mathbf{AB} \mid \mathbf{A} \mid \epsilon$$

$$A' \rightarrow \mathbf{AA''} \mid \epsilon$$

$$A'' \rightarrow \mathbf{B} \mid \epsilon$$

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \dots \dots \mid r_1 \mid r_2$$



$$A \rightarrow \alpha A' \mid r_1 \mid r_2$$

$$A' \rightarrow \beta_1 \mid \beta_2 \dots$$

Exercise

1. $S \rightarrow iEtS \mid iEtSeS \mid a$
2. $A \rightarrow ad \mid a \mid ab \mid abc \mid x$
3. $S \rightarrow aSSbS \mid aSaSb \mid abb \mid b$
4. $A \rightarrow aAB \mid aBc \mid aAc$
5. $S \rightarrow aAd \mid aB$
 $A \rightarrow a \mid ab$
 $B \rightarrow ccd \mid ddc$

1. $S \rightarrow iEtS \mid iEtSeS \mid a$

Soln:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow \epsilon \mid es$$

2. $A \rightarrow ad \mid a \mid ab \mid abc \mid x$

Soln:

$$A \rightarrow aA' \mid x$$

$$A' \rightarrow d \mid \epsilon \mid b \mid bc$$

$$A' \rightarrow bA''$$

$$A'' \rightarrow c \mid \epsilon$$

Removal of Ambiguity in Grammars

Elimination of left factoring

$$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \dots / \alpha\beta_n / \dots$$

$$A \rightarrow \alpha A' / \alpha_1 / \alpha_2$$

$$A' \rightarrow \beta_1 / \beta_2 / \dots$$

Examp^l:

$$(1) S \rightarrow i\epsilon\epsilon S / i\epsilon\epsilon S\epsilon s / a$$

$$\epsilon \rightarrow b$$

$$\frac{S}{A} \rightarrow \frac{i\epsilon\epsilon S}{\alpha} / \frac{i\epsilon\epsilon S\epsilon s}{\alpha} / \frac{a}{\beta_2 \quad \alpha_1}$$

$$\boxed{\begin{array}{l} S \rightarrow i\epsilon\epsilon S S' / a \\ S' \rightarrow \epsilon / \epsilon s \\ \epsilon \rightarrow b \end{array}}$$

$$(2) A \rightarrow aAB / aA / a$$

$$B \rightarrow bB / b$$

$$A \rightarrow \frac{aAB}{\alpha \quad \beta_1} / \frac{aA}{\alpha \quad \beta_2} / \frac{a}{\alpha}$$

$$A \rightarrow aA'$$

$$A' \rightarrow AB / A / \epsilon$$

$$B \rightarrow \frac{bB}{\alpha \quad \beta_1} / \frac{b}{\alpha}$$

$$B \rightarrow bB'$$

$$B' \rightarrow B / \epsilon$$

$$A \rightarrow aA'$$

$$A' \rightarrow AB / A / \epsilon$$

$$B \rightarrow bB'$$

$$B' \rightarrow B / \epsilon$$

$$(3) X \rightarrow X + X \mid X * X \mid D$$

$$D \rightarrow 1/2/3$$

$$X \rightarrow \frac{X+X}{\alpha \quad \beta_1} \mid \frac{X * X}{\alpha \quad \beta_2} \mid \frac{D}{\gamma_1}$$

$$\boxed{\begin{array}{l} X \rightarrow X X' \mid D \\ X' \rightarrow +X \mid *X \\ D \rightarrow 1/2/3 \end{array}}$$

$$(4) E \rightarrow T + E \mid T$$

$$T \rightarrow \text{cat} \mid \text{cat} * T \mid (E)$$

$$\frac{E}{A} \rightarrow \frac{T+E}{\alpha \quad \beta_1} \mid \frac{T}{\alpha}$$

$$E \rightarrow T E'$$

$$E' \rightarrow +E \mid \epsilon$$

$$T \rightarrow \frac{\text{cat}}{\alpha} \mid \frac{\text{cat} * T}{\alpha \quad \beta_1} \mid \frac{(E)}{\gamma_1}$$

$$T \rightarrow \text{cat} T' \mid (E)$$

$$T' \rightarrow *T \mid \epsilon$$

$$\boxed{\begin{array}{l} E \rightarrow T E' \\ E' \rightarrow +E \mid \epsilon \\ T \rightarrow \text{cat} T' \mid (E) \\ T' \rightarrow *T \mid \epsilon \end{array}}$$

Elimination of left recursion

Eliminate left recursion for the following grammar

$$(1) E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

$$\begin{array}{l} A \rightarrow A\alpha / B \\ \Downarrow \\ A \rightarrow BA' \\ A' \rightarrow \alpha A' / \epsilon \end{array}$$

$$\begin{array}{c} E \rightarrow E + T / T \\ A \quad A\alpha \quad B \end{array} \quad (\text{RHS and LHS contain same symbol (Non-terminal)})$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$\begin{array}{c} T \rightarrow T * F / F \\ A \quad A\alpha \quad B \end{array} \quad (\text{left recursion})$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (E) / id$$

$$\begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' / \epsilon \\ T \rightarrow FT' \\ T' \rightarrow *FT' / \epsilon \\ F \rightarrow (E) / id \end{array}$$

$$(2) S \rightarrow S_0 S_1 S / 01$$

$$\begin{array}{c} \underline{S} \rightarrow \underline{S_0 S_1 S} / 01 \\ A \quad A \quad \alpha \quad B \end{array}$$

$$\boxed{\begin{array}{l} S \rightarrow 01S' \\ S' \rightarrow 0S_1SS' / \epsilon \end{array}}$$

$$(3) S \rightarrow (L) / x$$

$$L \rightarrow L, S / s$$

$$\begin{array}{c} \underline{L} \rightarrow \underline{L, S} / \underline{S} \\ A \quad A \quad \alpha \quad B \end{array}$$

$$\boxed{\begin{array}{l} L \rightarrow SL' \\ L' \rightarrow , SL' / \epsilon \end{array}}$$

$$(4) \underline{A} \rightarrow \underline{AA} \alpha / B$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow A \alpha A' / \epsilon$$

$$(5) A \rightarrow ABd / Aa / a$$

$$B \rightarrow Be / b$$

$$\underline{A} \rightarrow \underline{ABd} / \underline{Aa} / \underline{a}$$

$$A \rightarrow aA'$$

$$A' \rightarrow BDA' / aA' / \epsilon$$

$$\underline{B} \rightarrow \underline{Be} / b$$

$$B \rightarrow bB'$$

$$B' \rightarrow eB' / \epsilon$$

$$(6) A \rightarrow ABd / Aa / a$$

$$B \rightarrow Be / b$$

$$\underline{A} \rightarrow \underline{ABd} / \underline{Aa} / \underline{a}$$

$$A \rightarrow aA'$$

$$A' \rightarrow bDA' / aA' / \epsilon$$

$$B \rightarrow bB'$$

$$B' \rightarrow eB' / \epsilon$$

$$(7) A \rightarrow AB / Ac / a / b$$

$$A \rightarrow aA' / bA'$$

$$A' \rightarrow BA' / CA' / \epsilon$$

$$(8) S \rightarrow A / B$$

$$A \rightarrow ABC / Acd / a / aa$$

$$B \rightarrow Bec / b$$

$$\underline{A} \rightarrow \underline{ABC} / \underline{Acd} / \underline{a} / \underline{aa}$$

$$A \rightarrow aA' / aaA'$$

$$A' \rightarrow BCA' / cdA' / \epsilon$$

$$\underline{B} \rightarrow \underline{Bec} / b$$

$$B \rightarrow bB'$$

$$B' \rightarrow eeB' / \epsilon$$

$$\boxed{\begin{array}{l} S \rightarrow A / B \\ A \rightarrow aA' / aaA' \\ A' \rightarrow BCA' / cdA' / \epsilon \\ B \rightarrow bB' \\ B' \rightarrow eeB' / \epsilon \end{array}}$$